

Travel Planner

A Multi-Agent AI System for Automated Trip Planning, Built with CrewAI

Author: Falak Baweja

Project Report — July 8, 2026

1 Overview

This project implements an automated travel-planning pipeline using **CrewAI**, a framework for orchestrating multiple large-language-model (LLM) agents that collaborate on a shared goal. Rather than relying on a single prompt to produce a trip plan, the system decomposes planning into seven specialized roles — preference analysis, destination research, transportation, accommodation and budgeting, itinerary construction, safety assessment, and final report generation. Each agent focuses on one dimension of the problem and passes its output downstream, mirroring how a real travel agency divides work among specialists.

2 System Architecture

The Crew is composed of six persistent agents (the Safety agent shares the pipeline but is invoked via its own task) driven by two underlying language models: a general-purpose reasoning model for structured planning output, and a tool-oriented model for agents that must call external search and scraping tools. Table 1 summarizes each agent’s role and primary responsibility.

Table 1: System Architecture: Agents and Responsibilities

Agent	Role	Primary Responsibility
Preference Analyst	Interprets traveler input	Builds a structured traveler profile (budget tier, interests, pace, constraints)
Destination Research Specialist	Explores the destination	Surfaces attractions, hidden gems, local events and activities via web search
Transportation Planning Specialist	Plans movement	Recommends flights, local transport and airport transfers, optimizing cost/time
Accommodation & Budget Specialist	Manages spend	Suggests lodging and allocates/optimizes the overall trip budget
Itinerary Generation Specialist	Synthesizes the plan	Merges upstream outputs into a day-by-day schedule
Safety & Risk Assessment Specialist	Assesses risk	Flags weather, advisories and logistical risks, with mitigation tips
Travel Report Generation Specialist	Final integrator	Consolidates every agent’s output into one traveler-facing dossier

3 Task Pipeline

Execution follows CrewAI’s sequential process: each task runs in order and its output becomes context for the tasks that follow, so later specialists reason over earlier findings rather than working in isolation.

- **Task 1 — Analyze Preferences:** Converts destination, budget, duration, traveler count, and interests into a structured traveler profile.
- **Task 2 — Research Destination:** Gathers attractions, hidden gems, local events, and interest-aligned activities.
- **Task 3 — Plan Transportation:** Proposes flights, local transport, and transfers optimized for cost and time.
- **Task 4 — Accommodation & Budget:** Recommends lodging and allocates spend within the stated budget.
- **Task 5 — Generate Itinerary:** Merges all prior outputs into a cohesive day-by-day schedule.
- **Task 6 — Safety Assessment:** Checks weather, travel advisories, and logistical risk, appending vital mitigation tips.
- **Task 7 — Final Report:** The Report Generation agent consolidates every upstream output into one comprehensive, traveler-facing dossier.

4 Technology Stack

The underlying implementation leverages a modular suite of tools and specific models optimized for cost, latency, and capability constraints.

Table 2: Technology Stack Breakdown

Component	Detail
Orchestration framework	CrewAI (<code>Process.sequential</code>) coordinating 6 agents across 7 tasks
Reasoning LLM	Groq-hosted <code>llama-3.3-70b-versatile</code> — used by agents that write structured, decision-heavy output
Tool-use LLM	Groq-hosted <code>openai/gpt-oss-120b</code> — used by agents that call search/scrape tools (Destination, Safety)
Web search	<code>SerperDevTool</code> — live search grounding for destination and safety research
Web scraping	<code>ScrapeWebsiteTool</code> — pulls page-level detail from sources found via search
Execution mode	Asynchronous kickoff (<code>crew.kickoff_async</code>) with a request-rate cap (<code>max_rpm = 4</code>)

5 Sample Run

The notebook’s demonstration run plans a trip to Switzerland using the following benchmark parameters:

Destination: Switzerland
Budget: 300,000 INR
Duration: 4 nights, 5 days
Travelers: 2
Interests: chocolate, adventure, photography

This single set of inputs is threaded through all seven tasks via CrewAI’s templated placeholders (e.g., `{destination}`, `{budget}`, `{duration}`, `{travelers}`, `{interests}`), proving that the same pipeline cleanly generalizes to any destination or traveler profile without code changes.

6 Design Notes & Considerations

- **Token Optimization:** Splitting the LLM by function (reasoning vs. tool-use) keeps token budgets tight. Agents that only reason use a 900-token cap, while tool-calling agents are capped lower at 450 tokens.

- **Rate Limiting:** `max_iter` limits (1–2 per agent) and `max_rpm = 4` on the Crew bound both cost and latency, which strictly matters given the free-tier rate limits typical of hosted LLM APIs like Groq.
- **Reproducibility:** Memory is disabled per agent (`memory=False`), ensuring each run is stateless and easily reproducible for a given input set.
- **Security:** API keys for Groq and Serper are read from environment variables at runtime and are never committed to source control or notebook outputs in plaintext.

7 Conclusion

This project demonstrates a practical pattern for multi-agent LLM systems: breaking a complex, multi-faceted task into narrow expert roles, chaining them with shared context, and letting a final synthesis agent produce a coherent deliverable. The architecture is readily extensible—additional specialists (e.g., visa/documentation, dining recommendations, currency conversion) could be inserted into the same sequential pipeline with minimal changes to the existing agents and tasks.